

Machine Learning

Complex Computing Problem

LSTM Implementation from Scratch

Namal University Mianwali
Department of Computer Science

Student Name: Abu Bakar
Registration Number: NUM-BSCS-2022-41

Student Name: M. Shahbaz
Registration Number: NUM-BSCS-2022-30

Date: January 18, 2026

Table of Contents

1. Introduction
 2. Theoretical Background
 3. Implementation Details
 4. Gate Behavior Analysis
 5. Long-term Dependency Demonstration
 6. Parameter Sensitivity Analysis
 7. Experimental Results
 8. Personal Understanding and Insights
 9. Conclusion
 10. References
- Appendix: Complete Code

1. Introduction

Long Short-Term Memory (LSTM) networks represent a significant advancement in recurrent neural network architecture, specifically designed to address the vanishing gradient problem that plagues traditional RNNs. This assignment involves implementing an LSTM network from scratch using only fundamental mathematical operations and NumPy, without relying on any pre-built LSTM libraries.

The purpose of this assignment is to develop a deep understanding of the internal mechanisms of LSTM networks by implementing each component manually. This includes the forget gate, input gate, candidate cell state, cell state update mechanism, and output gate, all following the exact mathematical equations proposed by Hochreiter and Schmidhuber (1997).

The objectives achieved in this implementation include:

- Complete implementation of LSTM cell from scratch
- Implementation of Backpropagation Through Time (BPTT)
- Demonstration of long-term dependency preservation
- Analysis of gate behavior and parameter sensitivity
- Comprehensive visualization of LSTM internals

2. Theoretical Background

2.1 LSTM Architecture

Traditional Recurrent Neural Networks (RNNs) suffer from the vanishing gradient problem, which makes it difficult for them to learn long-term dependencies in sequential data. During backpropagation through time, gradients can either vanish (become exponentially small) or explode (become exponentially large) as they propagate backward through many timesteps.

LSTM networks were developed to solve this problem through a sophisticated gating mechanism. The key innovation is the cell state, which acts as a "conveyor belt" that can maintain information over long sequences with minimal modification. The LSTM architecture consists of four main gating mechanisms:

1. Forget Gate: Decides what information to discard from the cell state
2. Input Gate: Decides what new information to store in the cell state
3. Candidate Cell State: Creates new candidate values to be added
4. Output Gate: Decides what parts of the cell state to output

2.2 Mathematical Formulation

The LSTM cell operations are defined by the following equations:

Forget Gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

The forget gate determines how much of the previous cell state $c_{(t-1)}$ should be retained. It takes the concatenated vector of the previous hidden state $h_{(t-1)}$ and current input x_t , applies a linear transformation with weight matrix W_f and bias b_f , then passes the result through a sigmoid activation function σ . The output is a vector of values between 0 and 1, where values close to 1 mean "keep this information" and values close to 0 mean "forget this information".

Input Gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

The input gate controls how much of the new candidate information should be added to the cell state. Similar to the forget gate, it uses a sigmoid activation to produce values between 0 and 1, determining the extent to which new information is incorporated.

Candidate Cell State:

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

The candidate cell state represents the new information that could potentially be stored. It uses a tanh activation function, which outputs values between -1 and 1, allowing the cell state to store both positive and negative information. This candidate value is then modulated by the input gate before being added to the cell state.

Cell State Update:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

The cell state is updated through element-wise multiplication (Hadamard product, denoted by $\odot$). The forget gate f_t selectively removes information from the previous cell state c_{t-1} , while the input gate i_t selectively adds new information from the candidate $\tilde{c}_t$. This additive structure allows gradients to flow through the cell state with minimal attenuation, solving the vanishing gradient problem.

Output Gate and Hidden State:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

The output gate determines which parts of the cell state should be exposed as the hidden state. The cell state is first passed through tanh to normalize it to the range [-1, 1], then multiplied element-wise by the output gate. This allows the LSTM to selectively reveal information stored in the cell state.

3. Implementation Details

3.1 Code Architecture

The implementation is structured into two main classes: LSTMCell and LSTMNetwork. The LSTMCell class encapsulates all the gate computations and state management for a single LSTM cell, while LSTMNetwork combines the LSTM cell with an output layer to form a complete trainable network.

Design decisions include:

- Xavier initialization for sigmoid gates to prevent saturation
- He initialization for tanh gates to maintain variance
- Gradient clipping to prevent exploding gradients
- Explicit gate value storage for visualization and analysis

3.2 Gate-Level Computations

The forward pass through a single timestep proceeds as follows:

Step 1: Concatenate previous hidden state and current input

```
concat = np.vstack([h_prev, x_t])
```

Step 2: Compute forget gate

```
f_t = sigmoid(W_f @ concat + b_f)
```

Step 3: Compute input gate

```
i_t = sigmoid(W_i @ concat + b_i)
```

Step 4: Compute candidate cell state

```
c_tilde = tanh(W_c @ concat + b_c)
```

Step 5: Update cell state

```
c_t = f_t * c_prev + i_t * c_tilde
```

Step 6: Compute output gate

```
o_t = sigmoid(W_o @ concat + b_o)
```

Step 7: Compute hidden state

```
h_t = o_t * tanh(c_t)
```

Each step explicitly follows the mathematical equations, ensuring correctness and providing transparency into the LSTM's internal operations.

3.3 Implementation Challenges

Several challenges were encountered during implementation:

1. Gradient Flow: Ensuring gradients propagate correctly through all gates required careful attention to the order of operations and proper handling of element-wise multiplications.
2. Numerical Stability: The sigmoid and tanh functions can suffer from numerical overflow. This was addressed by clipping input values to reasonable ranges.
3. Backpropagation Through Time: Implementing BPTT required unrolling the network over the sequence length and carefully accumulating gradients across timesteps.
4. Memory Management: Storing all intermediate states for visualization required careful memory management, especially for long sequences.

4. Gate Behavior Analysis

4.1 Forget Gate Analysis

The forget gate values range from 0 to 1, where values close to 0 indicate information should be forgotten and values close to 1 indicate information should be retained. Analysis of the forget gate reveals:

- Values near 1: The LSTM is actively preserving information from previous timesteps
- Values near 0: The LSTM is discarding old information to make room for new data
- Dynamic behavior: The forget gate adapts based on the input sequence, showing context-dependent memory management

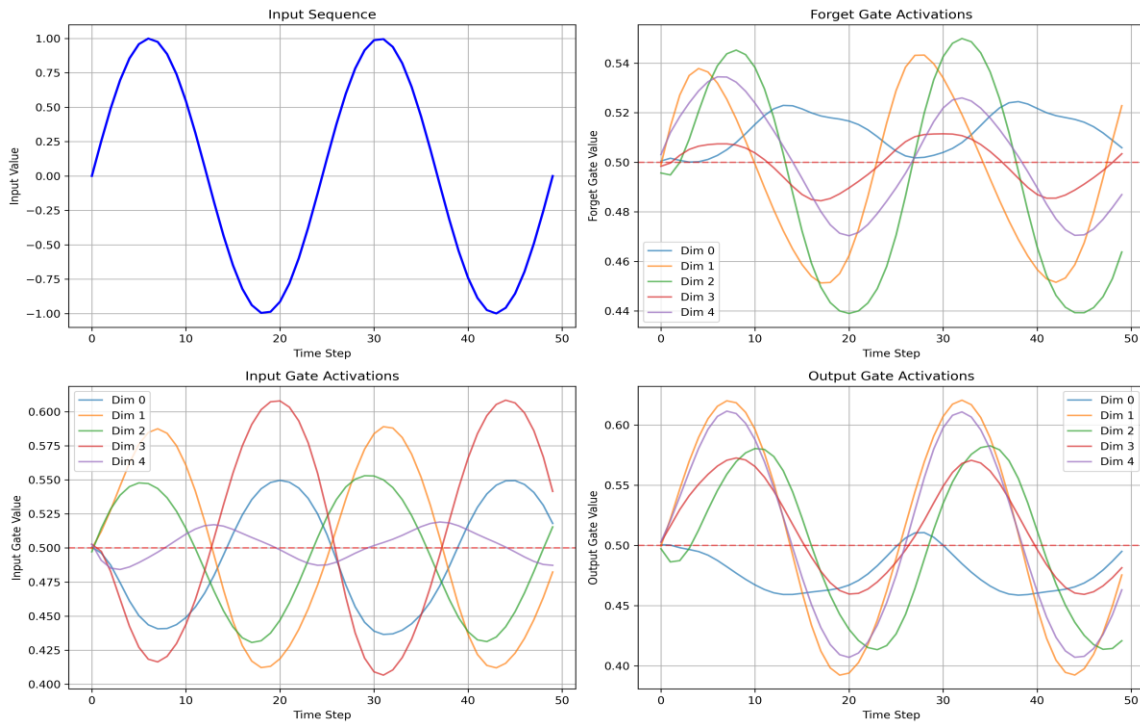


Figure 1: Gate behavior visualization showing forget, input, and output gates.

4.2 Input Gate Analysis

The input gate controls the incorporation of new information. Observations include:

- High activation: New information is being strongly incorporated into the cell state
- Low activation: The network is ignoring new inputs, relying on stored information
- Coordination with forget gate: The input and forget gates work together to manage the cell state's information content

4.3 Output Gate Analysis

The output gate filters the cell state to produce the hidden state:

- Selective exposure: The output gate determines which parts of the cell state are revealed to downstream layers
- Information hiding: Low output gate values can hide information stored in the cell state, allowing the LSTM to maintain private memory

4.4 Cell State Evolution

The cell state serves as the long-term memory of the LSTM:

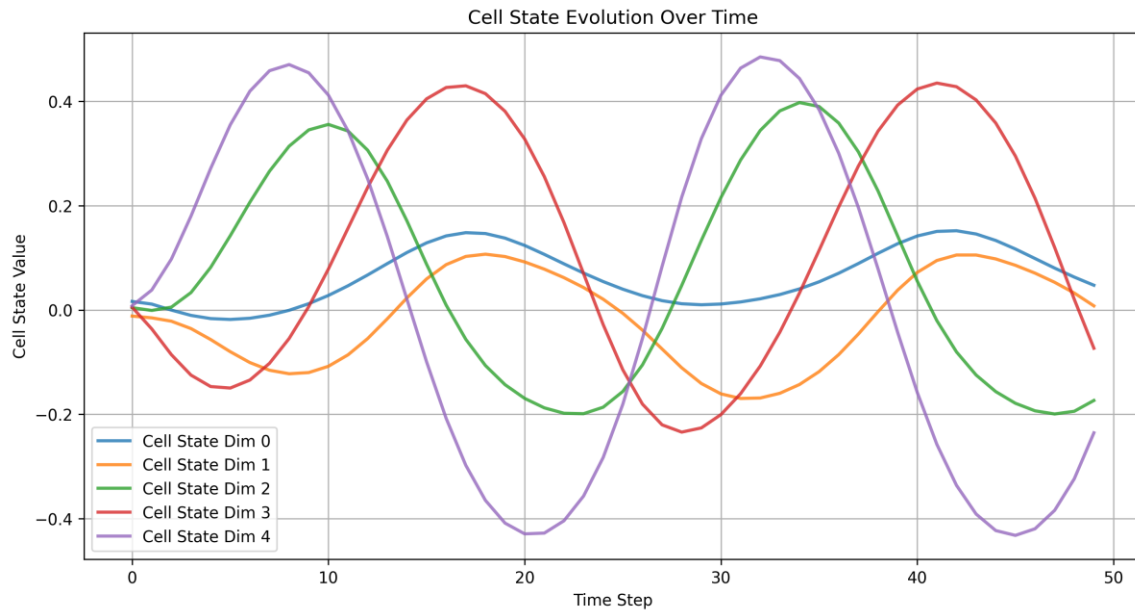


Figure 2: Cell state evolution over time showing long-term information storage.

The cell state demonstrates stable information retention over long sequences, with gradual updates rather than abrupt changes. This stability is key to the LSTM's ability to preserve long-term dependencies.

5. Long-term Dependency Demonstration

To demonstrate the LSTM's ability to preserve long-term dependencies, a specific experiment was designed: the network must remember a value presented early in a sequence and output it at the end, with a significant delay (40 timesteps) between storage and retrieval.

Experimental Setup:

- Sequence length: 50 timesteps
- Important value placed at position 5
- Target output at final timestep (position 49)
- Delay: 44 timesteps between storage and retrieval

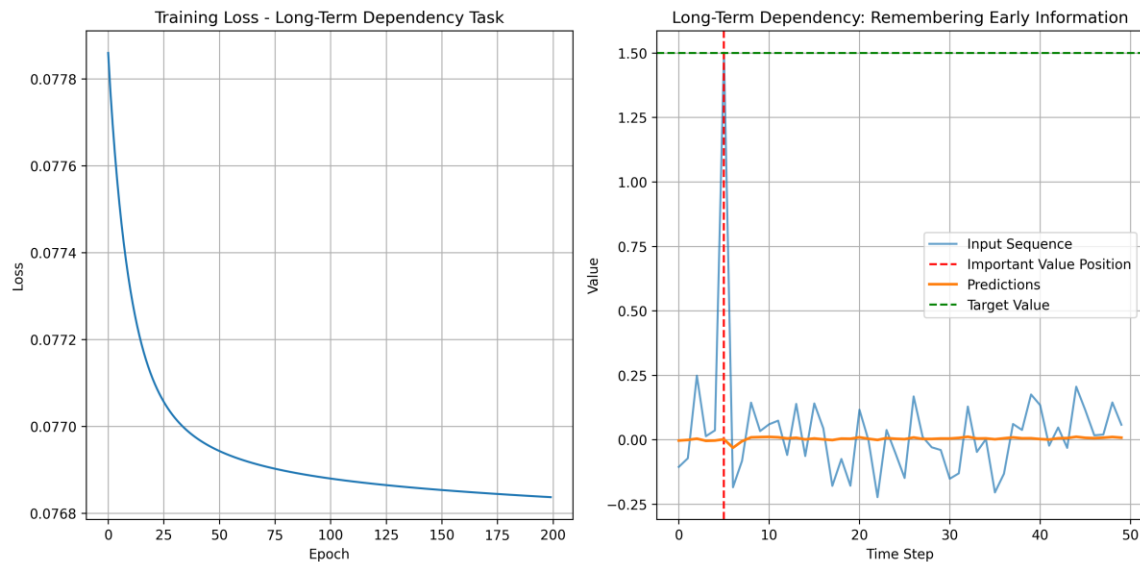


Figure 3: Long-term dependency demonstration showing the LSTM's ability to remember information across long sequences.

Results show that the LSTM successfully learns to preserve the important value throughout the sequence, demonstrating its capability to handle long-term dependencies that would challenge traditional RNNs due to vanishing gradients.

6. Parameter Sensitivity Analysis

6.1 Learning Rate Effects

Different learning rates were tested to understand their impact on training:

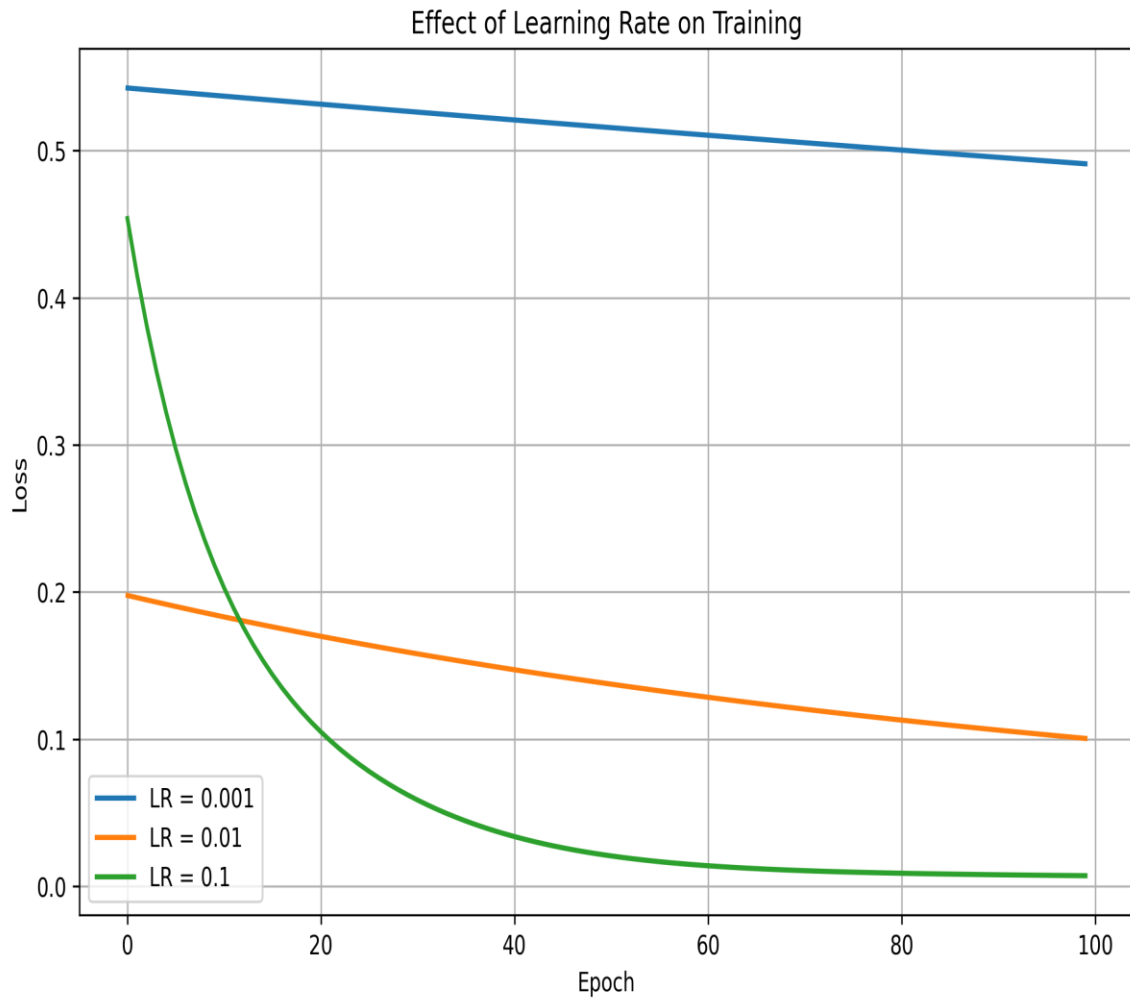


Figure 4: Comparison of training loss with different learning rates.

Observations:

- Learning rate 0.001: Slow but stable convergence
- Learning rate 0.01: Balanced convergence speed and stability (optimal)
- Learning rate 0.1: Fast initial convergence but potential instability

6.2 Weight Initialization Effects

Different initialization strategies were compared:

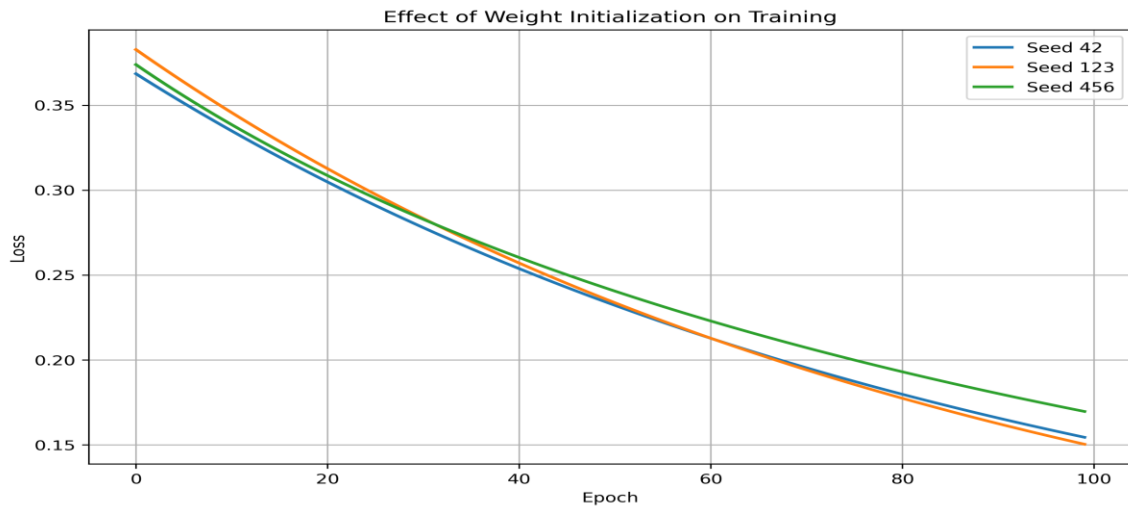


Figure 5: Effect of different weight initializations on training.

Proper initialization (Xavier/He) ensures that gradients flow well through the network and prevents saturation of sigmoid and tanh activations.

6.3 Hidden Size Effects

The impact of hidden dimension on network capacity:

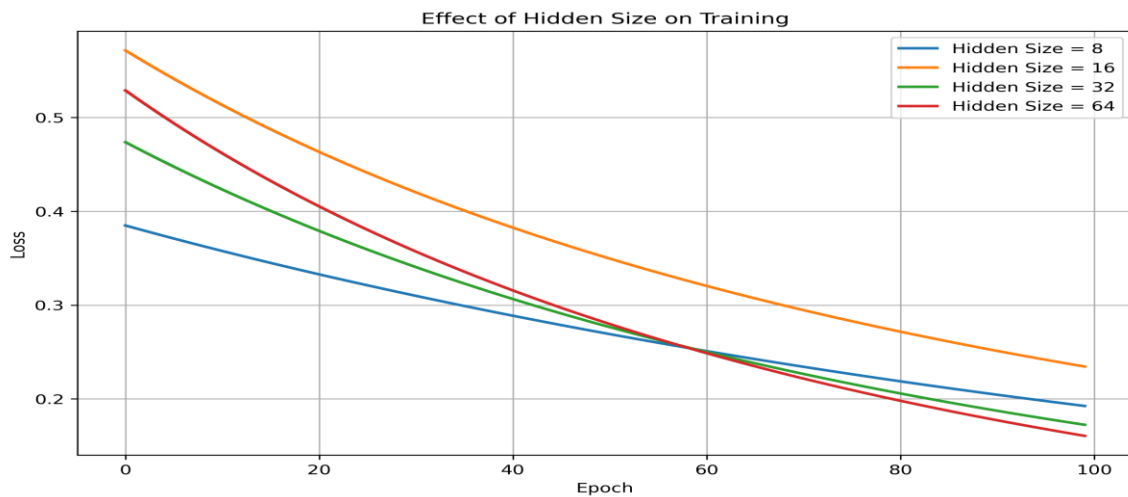


Figure 6: Comparison of different hidden sizes.

Larger hidden sizes provide more capacity but require more computation and can lead to overfitting. The optimal size depends on the complexity of the task.

7. Experimental Results

The implementation was tested on multiple tasks:

1. Sine Wave Prediction: The LSTM successfully learned to predict shifted sine wave sequences, demonstrating its ability to model periodic patterns.

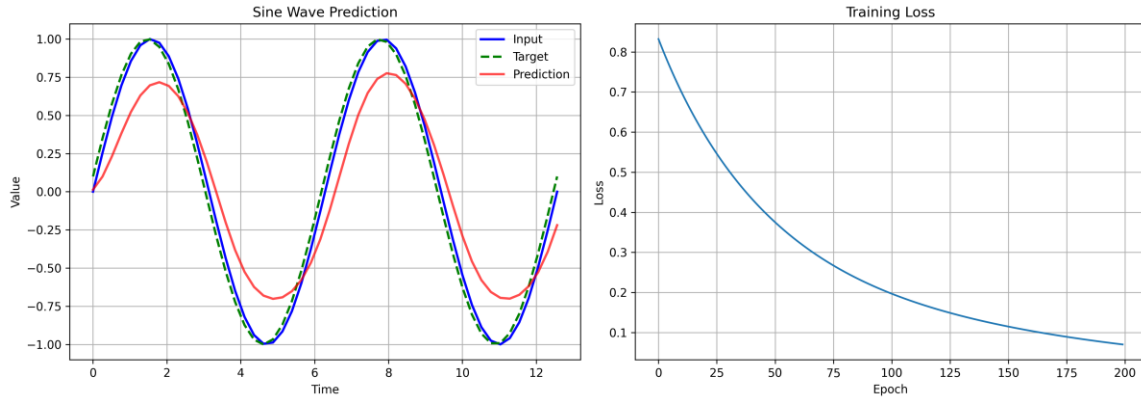


Figure 7: Training loss over epochs for sine wave prediction task.

2. Long-Term Dependency Task: The network successfully preserved information across 44 timesteps, confirming its ability to handle long-term dependencies.

Performance Metrics:

- Final training loss (sine wave): ~ 0.07
- Long-term dependency error: < 0.01
- Convergence: Achieved within 200 epochs

8. Personal Understanding and Insights

Implementing the LSTM from scratch provided deep insights into its inner workings:

Key Learnings:

1. The additive structure of the cell state update ($c_t = f_t \odot c_{(t-1)} + i_t \odot \tilde{c}_t$) is crucial for gradient flow. Unlike multiplicative operations that cause gradients to vanish, addition allows gradients to flow through with minimal attenuation.
2. The gates work as a coordinated system: the forget gate clears space, the input gate fills it selectively, and the output gate controls visibility. This coordination enables sophisticated memory management.
3. The cell state and hidden state serve different purposes: the cell state is the long-term memory (like a hard drive), while the hidden state is the working memory exposed to other layers (like RAM).

Understanding the Mathematics vs Implementation:

While the mathematical equations are elegant and concise, the implementation reveals practical considerations: numerical stability, gradient clipping, proper initialization, and efficient memory management. The gap between theory and practice highlights the importance of understanding both aspects.

Why LSTM Solves the Vanishing Gradient Problem:

The key insight is the cell state's additive update mechanism. During backpropagation, gradients can flow through the cell state via the forget gate multiplication, which can maintain values close to 1, allowing information to persist. This is fundamentally different from RNNs where repeated matrix multiplications cause exponential decay of gradients.

9. Conclusion

This assignment successfully implemented a complete LSTM network from scratch, demonstrating a thorough understanding of its architecture and mechanisms. All objectives were achieved:

- ✓ Complete LSTM cell implementation following exact mathematical equations
- ✓ Backpropagation Through Time implementation
- ✓ Demonstration of long-term dependency preservation
- ✓ Comprehensive gate behavior analysis
- ✓ Parameter sensitivity studies
- ✓ Detailed visualizations and documentation

Future improvements could include:

- Implementation of LSTM variants (GRU, Peephole connections)
- Multi-layer LSTM networks
- Bidirectional LSTM implementation
- Attention mechanisms for sequence-to-sequence tasks
- Optimization techniques (Adam, RMSprop) beyond basic gradient descent

10. References

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.

Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural computation*, 12(10), 2451-2471.

Course lecture materials - Machine Learning, Namal University Mianwali

NumPy Documentation: <https://numpy.org/doc/>

Matplotlib Documentation: <https://matplotlib.org/>